

Guía Definitiva de Comandos SQL para PostgreSQL

🎓 NIVEL PRINCIPIANTE

🗄️ POSTGRESQL

Sistema de gestión de bases de datos relacional, potente y de código abierto. Ideal para almacenar, organizar y recuperar datos de forma segura y eficiente.

¿Por qué PostgreSQL?

- Confiable y estable — usado por empresas Fortune 500
- Gratuito y de código abierto
- Soporta datos complejos (JSON, arrays, tipos personalizados)
- Excelente rendimiento y seguridad
- Comunidad activa y documentación completa

Estructura de este curso

Aprenderás los **4 pilares de SQL** organizados en una progresión lógica de menor a mayor complejidad:



Comandos DDL — CREATE TABLE

DDL (Data Definition Language) define la estructura de las tablas — el esqueleto sobre el que viven tus datos. Es el primer paso en cualquier proyecto de base de datos.

Sintaxis básica

```
CREATE TABLE nombre_tabla (  
  columna1 tipo_dato,  
  columna2 tipo_dato,  
  columna3 tipo_dato  
);
```

Ejemplo práctico

```
CREATE TABLE usuarios (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100),  
  email VARCHAR(100),  
  edad INT,  
  fecha_registro DATE  
);
```

PRIMARY KEY — Identifica cada fila de forma única. Cada tabla debe tener una. **SERIAL** la genera automáticamente.

Tipos de datos comunes

SERIAL

Entero que se incrementa automáticamente. Ideal para IDs únicos.

VARCHAR(n)

Texto de hasta **n** caracteres. Ej: `VARCHAR(100)` para nombres.

INT

Número entero. Para edades, cantidades, contadores.

DECIMAL(p,s)

Número decimal. Ej: `DECIMAL(10,2)` para valores monetarios.

DATE

Fecha en formato `YYYY-MM-DD`. Para registros temporales.

BOOLEAN

Verdadero o falso. Para flags, estados activo/inactivo.

Comandos DDL — ALTER TABLE y DROP TABLE

Una vez creada una tabla, necesitarás modificar su estructura con el tiempo. **ALTER TABLE** permite hacerlo de forma segura. **DROP TABLE** la elimina por completo.

ALTER TABLE — Modificar la estructura

Agregar una columna:

```
ALTER TABLE usuarios  
ADD COLUMN telefono VARCHAR(15);
```

Eliminar una columna:

```
ALTER TABLE usuarios  
DROP COLUMN telefono;
```

Cambiar el tipo de dato:

```
ALTER TABLE usuarios  
ALTER COLUMN edad TYPE VARCHAR(3);
```

DROP TABLE — Eliminar una tabla

```
-- Eliminar directamente  
DROP TABLE usuarios;
```

```
-- Opción segura: verificar si existe  
DROP TABLE IF EXISTS usuarios;
```

⊗ ⚠ **ADVERTENCIA CRÍTICA:** `DROP TABLE` elimina la tabla y **TODOS** sus datos permanentemente. No hay vuelta atrás. Usa siempre `IF EXISTS` para evitar errores si la tabla no existe.

Resumen de ALTER TABLE

ADD COLUMN

Agrega una nueva columna a la tabla existente.

DROP COLUMN

Elimina una columna y todos sus datos.

ALTER COLUMN

Cambia el tipo de dato de una columna.

Comandos DML — INSERT INTO

DML (Data Manipulation Language) es el lenguaje que te permite insertar, modificar y eliminar los datos que viven dentro de tus tablas.

Sintaxis básica

```
INSERT INTO tabla (columna1, columna2)
VALUES (valor1, valor2);
```

Insertar un registro

```
INSERT INTO usuarios
(nombre, email, edad, fecha_registro)
VALUES
('Juan Pérez', 'juan@email.com',
28, '2024-01-15');
```

 El campo **id (SERIAL)** se genera automáticamente. No necesitas especificarlo en el INSERT.

Insertar múltiples registros a la vez

```
INSERT INTO usuarios
(nombre, email, edad, fecha_registro)
VALUES
('María López', 'maria@email.com',
32, '2024-01-16'),
('Carlos Ruiz', 'carlos@email.com',
25, '2024-01-17'),
('Ana García', 'ana@email.com',
29, '2024-01-18');
```

Insertar múltiples filas en un solo comando es más eficiente que ejecutar un `INSERT` por cada registro. PostgreSQL procesa todo el lote en una sola operación.

1

Especifica columnas

Lista explícitamente las columnas destino para evitar errores de orden.

2

Separa con comas

Cada conjunto de `VALUES` se separa con una coma.

3

Textos entre comillas

Los valores de texto siempre van entre comillas simples
`'...'`.

Comandos DML — UPDATE

UPDATE modifica datos ya existentes en una tabla. Es uno de los comandos más poderosos — y más peligrosos — de SQL. El uso correcto de **WHERE** es absolutamente crítico.

Sintaxis

```
UPDATE tabla  
SET columna = valor  
WHERE condición;
```

Ejemplo — Actualizar la edad

```
UPDATE usuarios  
SET edad = 30  
WHERE nombre = 'Juan Pérez';
```

Actualizar múltiples columnas

```
UPDATE usuarios  
SET edad = 31,  
    email = 'juan.nuevo@email.com'  
WHERE id = 1;
```

Separa las columnas a actualizar con comas dentro del **SET**. Siempre preferible usar el **id** en el **WHERE** para mayor precisión.

⚠ CRÍTICO: El WHERE en UPDATE

Sin **WHERE**, actualizarás **TODOS** los registros de la tabla. Este es el error más común y más costoso para principiantes.

✗ INCORRECTO — actualiza TODOS los usuarios:

```
UPDATE usuarios  
SET edad = 30;
```

✓ CORRECTO — actualiza solo a Juan (id=1):

```
UPDATE usuarios  
SET edad = 30  
WHERE id = 1;
```

Regla de oro: Antes de ejecutar un **UPDATE**, escribe primero el **SELECT** equivalente con el mismo **WHERE** para verificar qué filas serán afectadas.

Comandos DML — DELETE vs TRUNCATE

Existen dos formas de eliminar datos: **DELETE** para registros específicos y **TRUNCATE** para vaciar una tabla completa. Conocer la diferencia es esencial para usarlos correctamente.

DELETE — Eliminar registros específicos

```
DELETE FROM tabla WHERE condición;
```

Eliminar un usuario por ID:

```
DELETE FROM usuarios WHERE id = 3;
```

Eliminar múltiples registros con condición:

```
DELETE FROM usuarios WHERE edad < 18;
```

TRUNCATE — Vaciar toda la tabla

```
TRUNCATE TABLE usuarios;
```

⊗ ⚠ **Ambos son destructivos.** Siempre realiza un backup antes de ejecutar DELETE masivos o TRUNCATE.

Diferencias clave

Característica	DELETE	TRUNCATE
Velocidad	Más lento (fila por fila)	Muy rápido (bulk)
Permite WHERE	✓ Sí	✗ No
Resetea IDs	Con RESTART IDENTITY	Automáticamente
Uso típico	Borrado selectivo	Limpiar tabla entera
Reversible	Con transacción activa	No en la mayoría de casos

Usa **DELETE** cuando necesites precisión. Usa **TRUNCATE** solo cuando quieras eliminar absolutamente todo el contenido de una tabla.

Comandos DQL — **SELECT** Básico

DQL (Data Query Language) es el lenguaje para consultar y recuperar datos. **SELECT** es el comando más utilizado en SQL — dominar sus variantes es fundamental para cualquier desarrollador.

Sintaxis básica

```
SELECT columna1, columna2  
FROM tabla;
```

Ver todos los datos (todas las columnas)

```
SELECT * FROM usuarios;
```

El asterisco `*` significa *"todas las columnas"*. Útil para exploración rápida.

Seleccionar columnas específicas

```
SELECT nombre, email FROM usuarios;
```

Consejo de producción: Usa **SELECT** con columnas específicas en entornos reales. Evitar `SELECT *` es más rápido, consume menos red y hace el código más mantenible.

Alias — Renombrar columnas en el resultado

```
SELECT  
  nombre AS "Nombre Completo",  
  email AS "Correo Electrónico"  
FROM usuarios;
```

Los alias con **AS** no cambian la tabla — solo afectan cómo aparece el resultado en pantalla. Son muy útiles para reportes y vistas.

Buenas prácticas con SELECT

Nombra tus columnas explícitamente

Lista cada columna que necesitas en lugar de usar `*`.

Usa alias descriptivos

Facilita la lectura de resultados en aplicaciones y reportes.

Combina con WHERE

Filtra desde el principio para obtener solo lo que necesitas.

Comandos DQL — Filtrado con WHERE

La cláusula **WHERE** filtra las filas que devuelve un **SELECT**. Es la herramienta más poderosa para obtener exactamente los datos que necesitas — sin traer información innecesaria.

Sintaxis

```
SELECT * FROM tabla WHERE condición;
```

Operadores relacionales

Operador	Significado
=	Igual a
!= / <>	No igual a
> / <	Mayor que / Menor que
>= / <=	Mayor o igual / Menor o igual

```
-- Usuarios mayores de 25 años
SELECT * FROM usuarios WHERE edad > 25;
```

Operadores lógicos

AND

Ambas condiciones deben ser verdaderas.

OR

Al menos una condición debe ser verdadera.

NOT

Invierte la condición especificada.

Ejemplo con AND

```
SELECT * FROM usuarios
WHERE edad > 25
AND nombre = 'Juan Pérez';
```

Ejemplo con OR

```
SELECT * FROM usuarios
WHERE edad < 20
OR edad > 60;
```

Puedes combinar **AND** y **OR** usando paréntesis para controlar la precedencia: `WHERE (edad > 25 AND ciudad = 'Madrid') OR vip = true`.

Comandos DQL — ORDER BY y LIMIT

Una vez que filtras tus datos con **WHERE**, necesitas controlar *cómo se presentan*. **ORDER BY** ordena los resultados y **LIMIT** controla cuántos se devuelven — esenciales para paginación y rendimiento.

ORDER BY — Ordenar resultados

```
SELECT * FROM tabla
ORDER BY columna ASC | DESC;
```

ASC	DESC
Ascendente (A→Z, 0→9). Es el orden por defecto.	Descendente (Z→A, 9→0).

Ordenar por edad (menor a mayor):

```
SELECT * FROM usuarios
ORDER BY edad ASC;
```

Ordenar por nombre (Z a A):

```
SELECT * FROM usuarios
ORDER BY nombre DESC;
```

LIMIT — Limitar la cantidad de resultados

```
-- Primeros 5 usuarios
SELECT * FROM usuarios LIMIT 5;
```

LIMIT + OFFSET — Paginación

```
-- Página 3 (registros 11 al 15)
SELECT * FROM usuarios
LIMIT 5 OFFSET 10;
```

OFFSET indica cuántos registros saltar antes de comenzar a devolver resultados. Con `LIMIT 5 OFFSET 10` obtienes los registros 11 al 15.

Combinación completa

```
SELECT nombre, email, edad
FROM usuarios
WHERE edad > 18
ORDER BY edad DESC
LIMIT 10;
```

 El orden de las cláusulas importa: **SELECT → FROM → WHERE → ORDER BY → LIMIT**. PostgreSQL rechaza otras ordenaciones.

Buenas Prácticas y Checklist del Principiante

✓ Buenas Prácticas SQL

- **Siempre usa WHERE en UPDATE y DELETE**
Evita modificar o eliminar todos los registros accidentalmente. Es el error más frecuente y costoso.
- **Especifica columnas en SELECT**
Evita `SELECT *` en producción. Es más rápido, consume menos memoria y hace el código más claro.
- **Usa PRIMARY KEY en cada tabla**
Identifica filas de forma única y facilita las relaciones entre tablas.
- **Haz backup antes de operaciones destructivas**
`DROP`, `DELETE` masivo y `TRUNCATE` no tienen vuelta atrás sin backup.
- **Usa nombres descriptivos**
Tablas y columnas claras facilitan el mantenimiento a largo plazo.
- **Usa transacciones para operaciones críticas**
Usa `BEGIN`, `COMMIT` y `ROLLBACK` para agrupar operaciones y poder revertirlas.

Checklist para Principiantes

- ☐ Instalé PostgreSQL y puedo conectarme
- ☐ Creé mi primera tabla con `CREATE TABLE`
- ☐ Inserté datos con `INSERT INTO`
- ☐ Consulté datos con `SELECT` y `WHERE`
- ☐ Modifiqué datos con `UPDATE` (con `WHERE`)
- ☐ Eliminé datos con `DELETE` (con `WHERE`)
- ☐ Ordené resultados con `ORDER BY`
- ☐ Limité resultados con `LIMIT`
- ☐ Practiqué combinaciones de comandos
- ☐ Entiendo la diferencia entre DDL, DML y DQL

Próximos pasos

1

JOINS

Combina datos de múltiples tablas.

2

Agregaciones

`COUNT`, `SUM`, `AVG`, `GROUP BY`.

3

Índices

Optimiza la velocidad de consultas.

4

Subconsultas

Consultas dentro de consultas.